LABORATORY EXPERIMENT TESTS

AGTI (SEMESTER 5)

INFORMATION SECURITY

(IT-DP 3203)

SR.	JOBS	DATE	MARK GIVEN	MARK GOT
1.	Password Strength Checker (Test 1)		5	
2.	User Authentication Simulation (Test 2)		5	
Total			10	

Student Name: Ma Shar Thet Han

Roll Number: 3GIT-8

Sign:

Teacher Name:

Designation:

Sign:

1. PASSWORD STRENGTH CHECKER (TEST 1)

AIM: To develop a C++ program that evaluates the strength of a user-entered password based on predefined security criteria.

DESCRIPTION: Weak passwords are a major cause of security breaches in computer systems. This experiment evaluates the strength of a user-entered password by analyzing its characteristics and providing appropriate feedback. The program is written in C++ and executed in Code::Blocks, with input and output handled via the console.

ALGORITHM (Step-by-Step Logic)

1. Start the program.
2. Prompt the user to enter a password.
3. Check whether the password:
   - Contains uppercase letters
   - Contains lowercase letters
   - Contains digits
   - Contains special characters
   - Meets the minimum length requirement
4. Count the number of conditions satisfied.
5. Classify the password strength as:
   - Weak → If 1–2 conditions are satisfied
   - Moderate → If 3 conditions are satisfied
   - Strong → If 4 or more conditions are satisfied and the password length is at least 8 characters
6. Display the password strength to the user.
7. End the program.

PROGRAM:

```
#include <iostream>
#include <string>
```

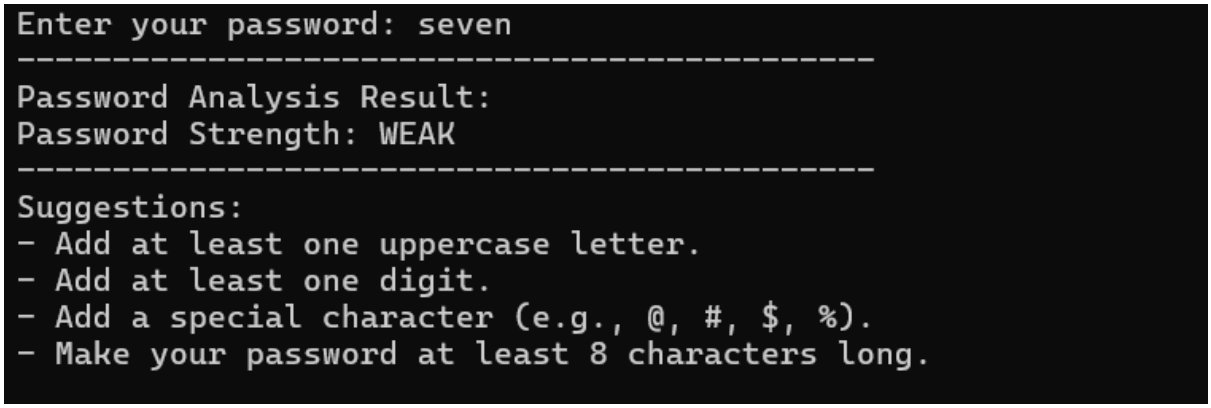
```
#include <cctype>

using namespace std;

int main() {
    string password;

    // Flags to track password conditions
    bool hasUppercase = false;
    bool hasLowercase = false;
    bool hasDigit = false;
    bool hasSpecialChar = false;

    // Prompt user for password
    cout << "Enter your password: ";
    getline(cin, password);

    // Analyze each character in the password
    for (char ch : password) {
        if (isupper(ch)) {
            hasUppercase = true;
        } else if (islower(ch)) {
            hasLowercase = true;
        } else if (isdigit(ch)) {
            hasDigit = true;
        } else {
            hasSpecialChar = true;
        }
    }

    // Count satisfied conditions
    int conditionCount = 0;
    if (hasUppercase) conditionCount++;
    if (hasLowercase) conditionCount++;
```

```
if (hasDigit) conditionCount++;
if (hasSpecialChar) conditionCount++;

// Display analysis header
cout << "-------------------------------------------\n";
cout << "Password Analysis Result:\n";

// Determine password strength
string strength;
if (conditionCount <= 2) {
    strength = "WEAK";
} else if (conditionCount == 3) {
    strength = "MODERATE";
} else if (conditionCount >= 4 && password.length() >= 8) {
    strength = "STRONG";
} else {
    strength = "MODERATE";
}

cout << "Password Strength: " << strength << endl;
cout << "-------------------------------------------\n";

// Display suggestions if password is not strong
cout << "Suggestions:\n";

if (strength == "STRONG") {
    cout << "(No suggestions - password is strong)\n";
} else {
    if (!hasUppercase)
        cout << "- Add at least one uppercase letter.\n";
    if (!hasLowercase)
        cout << "- Add at least one lowercase letter.\n";
    if (!hasDigit)
        cout << "- Add at least one digit.\n";
```

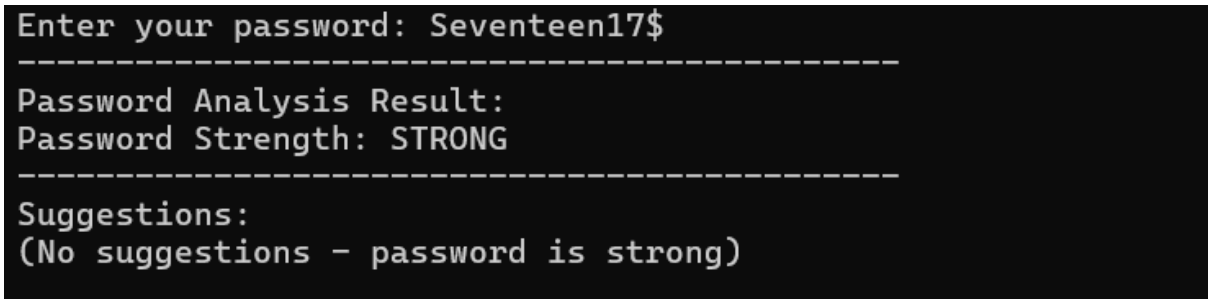
```
        if (!hasSpecialChar)
            cout << "- Add a special character (e.g., @, #, $, %).\n";
        if (password.length() < 8)
            cout << "- Make your password at least 8 characters long.\n";
    }
    return 0;
}
```

OUTPUT:

- Password Strength: WEAK



- Password Strength: STRONG



RESULT:

The C++ program was successfully executed and accurately evaluated the strength of the user-entered password.

Marking Scheme:

Sr.	Description	Mark Given	Mark Got

1.	Follow instructions correctly	1	
2.	List the algorithm	1	
3.	Produce expected output	2	
4.	Complete within time	1	

2. AUTHENTICATION SIMULATION (TEST 2)

AIM: To demonstrate how users are verified before gaining access to a system using a C++ program.

DESCRIPTION: Develop a C++ console program to simulate a user authentication system that validates login credentials using hashed passwords and a 6-digit OTP. The program grants access if both the password and OTP are correct, or displays appropriate error messages for invalid credentials. This exercise demonstrates username/password authentication, OTP verification, secure password handling, and console-based input/output. The program is implemented in C++ and executed in Code::Blocks, with all interactions performed via the console.

ALGORITHM (Step-by-Step Logic)

1. Start the program.
2. Seed the random number generator using the current time.
3. Create a user database (unordered map) to store usernames and hashed passwords.
4. Create a hash function for password hashing.

User Registration

5. Display "User Registration" prompt.
6. Ask the user to enter a username.
7. Check if the username already exists in the database:
   - If yes, display "Username already exists" and terminate the program.
8. Ask the user to enter a password.
9. Hash the password and store it in the database with the username.
10. Display "Registration successful".

User Login

11. Display "User Login" prompt.

12. Ask the user to enter username and password.
13. Check if the username exists in the database:
    - If not, display "Username not found" and terminate the program.
14. Hash the entered password and compare it with the stored hash:
    - If it does not match, display "Invalid password" and terminate the program.

OTP Verification

15. Generate a random 6-digit OTP.
16. Display the OTP (simulated) to the user.
17. Prompt the user to enter the OTP.
18. Compare the entered OTP with the generated OTP:
    - If it matches, display "Access Granted".
    - If it does not match, display "OTP verification failed" and "Access Denied".
19. End the program.

PROGRAM:

```
#include <iostream>
#include <string>
#include <unordered_map>
#include <cstdlib>
#include <ctime>
#include <functional>

using namespace std;

/* Function: generateOTP
   Purpose : Generates a random 6-digit OTP */
int generateOTP() {
   return 100000 + rand() % 900000;
}

int main() {
   // Seed random number generator
```

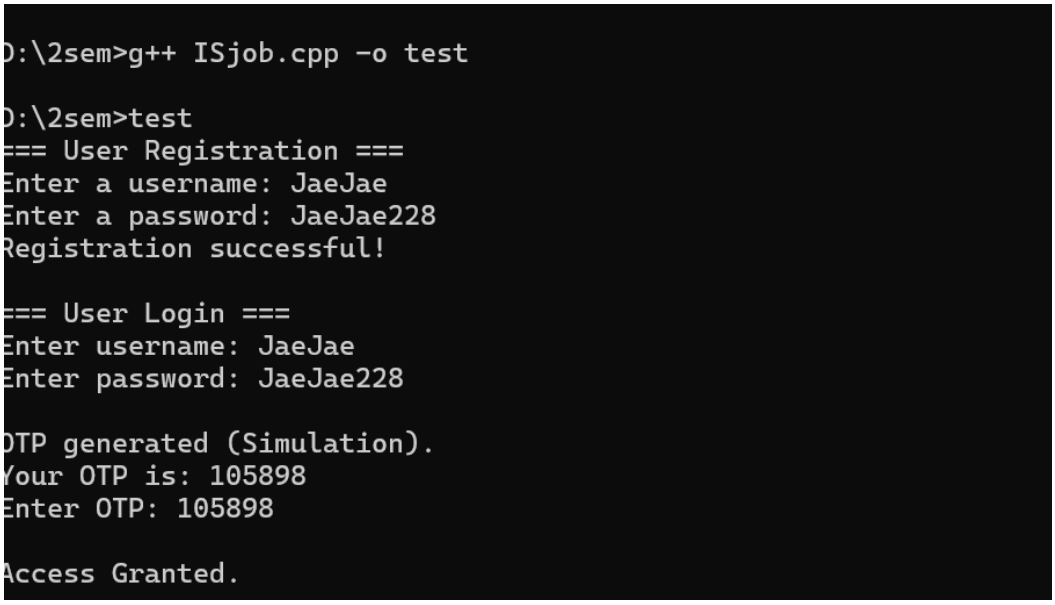
```
srand(static_cast<unsigned int>(time(0)));

// User database: username -> hashed password
unordered_map<string, size_t> userDatabase;

// Hash function for passwords
hash<string> hashPassword;

string username, password;

// USER REGISTRATION

cout << "=== User Registration ===\n";
cout << "Enter a username: ";
cin >> username;

// Check if username already exists
if (userDatabase.find(username) != userDatabase.end()) {
    cout << "Username already exists. Registration failed.\n";
    return 0;
}

cout << "Enter a password: ";
cin >> password;

// Store hashed password
userDatabase[username] = hashPassword(password);

cout << "Registration successful!\n\n";

cout << "=== User Login ===\n";
string loginUsername, loginPassword;

cout << "Enter username: ";
```

```
cin >> loginUsername;

cout << "Enter password: ";
cin >> loginPassword;

// Check if username exists
if (userDatabase.find(loginUsername) == userDatabase.end()) {
    cout << "\nUsername not found.\n";
    cout << "Access Denied.\n";
    return 0;
}

// Verify password
size_t enteredPasswordHash = hashPassword(loginPassword);

if (enteredPasswordHash != userDatabase[loginUsername]) {
    cout << "\nInvalid password.\n";
    cout << "Access Denied.\n";
    return 0;
}

// ========================
// OTP VERIFICATION
// ========================
int otp = generateOTP();

cout << "\nOTP generated (Simulation).\n";
cout << "Your OTP is: " << otp << endl;

int userOTP;
cout << "Enter OTP: ";
cin >> userOTP;

if (userOTP == otp) {
```

```
        cout << "\nAccess Granted.\n";
    } else {
        cout << "\nOTP verification failed.\n";
        cout << "Access Denied.\n";
    }
    return 0;
}
```

OUTPUT:

- Access Granted.



- Access Denied.

```
D:\2sem>g++ ISjob.cpp -o test

D:\2sem>test
=== User Registration ===
Enter a username: Shar Thet Han
Enter a password: Registration successful!

=== User Login ===
Enter username: Enter password: 1997

Username not found.
Access Denied.

D:\2sem>
```

RESULT:

The program was successfully executed and correctly verified user credentials. It generated and validated OTPs as expected, granting access only when both the password and OTP were correct.

Marking Scheme:

Sr.	Description	Mark Given	Mark Got
1.	Follow instructions correctly	1	
2.	List the algorithm	1	
3.	Produce expected output	2	
4.	Complete within time	1	